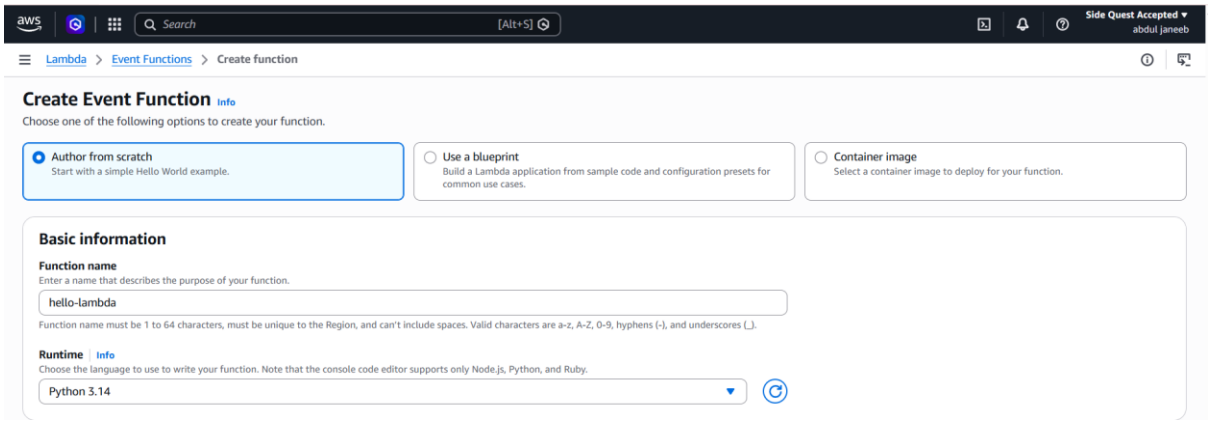
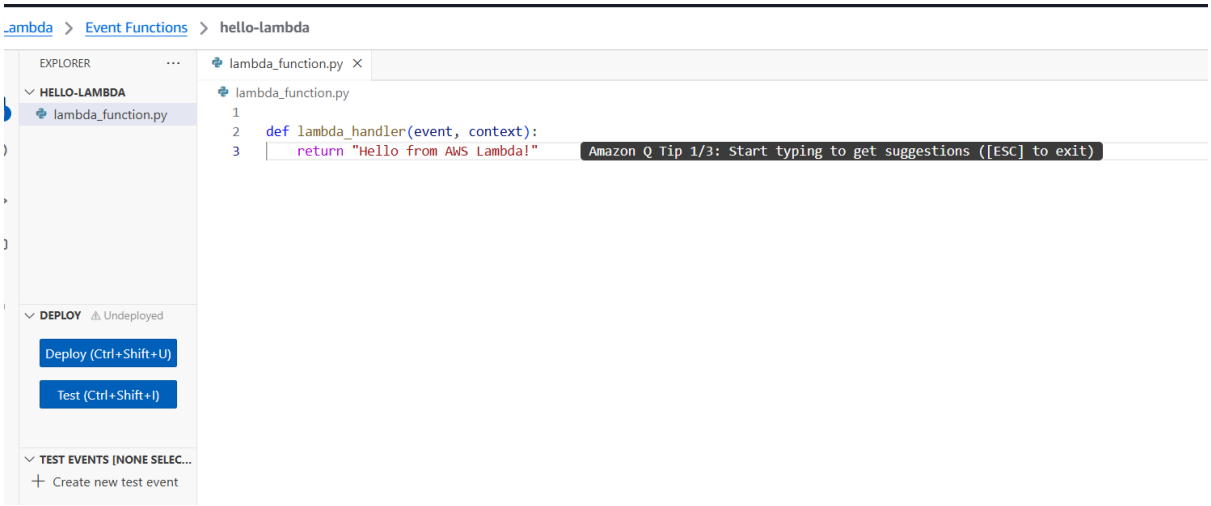
ABDUL JANEEB K

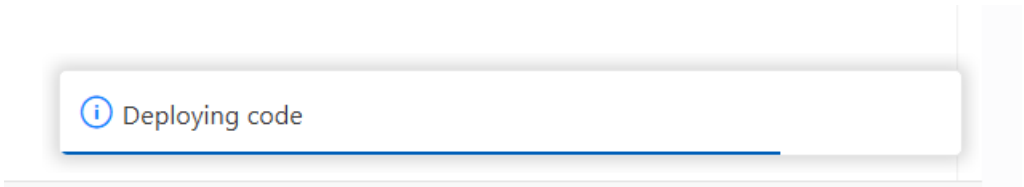
AWS Lambda Lab Exercises

- Created Lambda function

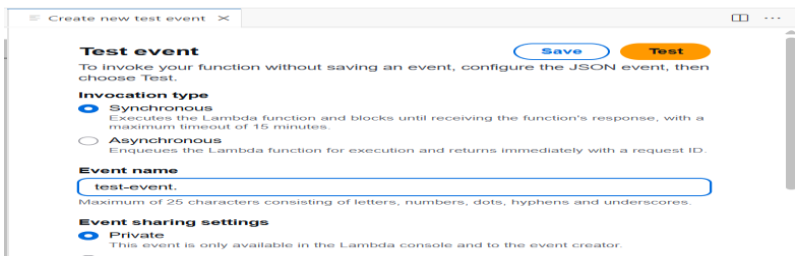




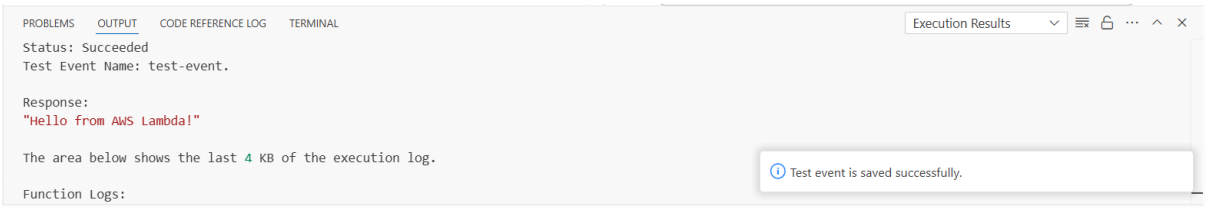
- After that deployed the code



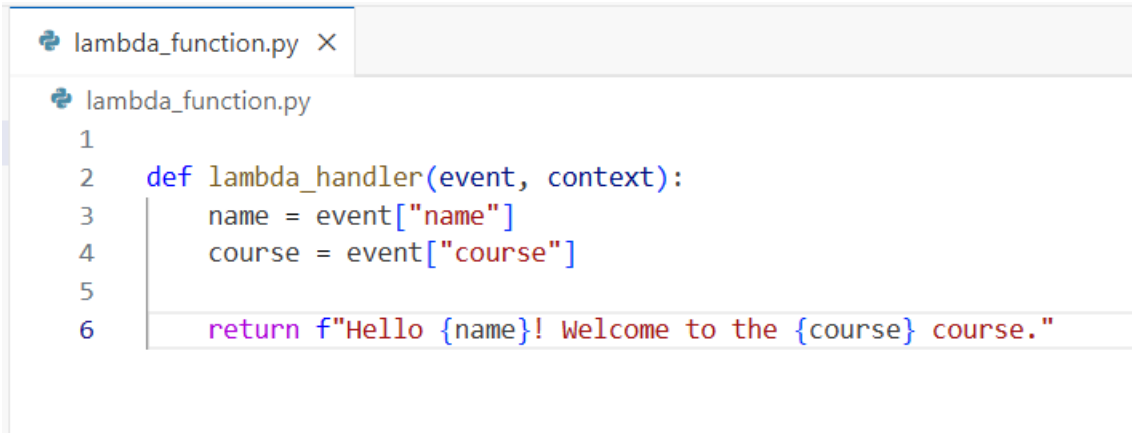
- After that I tested the function



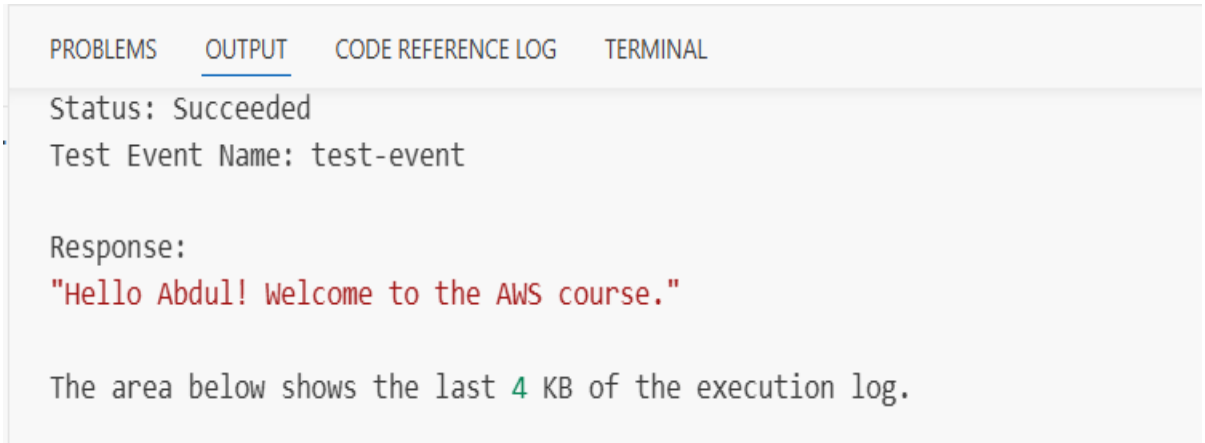
- Successfully executed



- Test event containing name and course







- Testing with different values

```
PROBLEMS  OUTPUT  CODE REFERENCE LOG  TERMINAL
Status: Succeeded
Test Event Name: test-event

Response:
"Hello janeeb! Welcome to the AWS course."

The area below shows the last 4 KB of the execution log.

Function Logs:
```

- Creating environment variable GREETING and INSTITUTE

lambda_function.py

```python
import os

def lambda_handler(event, context):
    name = event["name"]
    course = event["course"]

    greeting = os.environ["GREETING"]
    institute = os.environ["INSTITUTE"]

    return f"{greeting} {name}! Welcome to {course} at {institute}."
```

Event JSON

```
{
  "name": "Janeeb",
  "course": "AWS"
}
```

Edit environment variables

Environment variables

You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. Learn more

Key	Value	
GREETING	Welcome	Remove
INSTITUTE	ipsr solution	Remove

Add environment variable

▸ Encryption configuration

Cancel Save

- Successfully executed

```
PROBLEMS  OUTPUT  CODE REFERENCE LOG  TERMINAL
Status: Succeeded
Test Event Name: test-event

Response:
"Welcome Janeeb! Welcome to AWS at ipsr solution."

The area below shows the last 4 KB of the execution log.

Function Logs:
```

- **Calculator**

lambda_function.py

```python

def lambda_handler(event, context):
    a = float(event["num1"])
    b = float(event["num2"])

    result = {
        "addition": a + b,
        "subtraction": a - b,
        "multiplication": a * b,
        "division": a / b if b != 0 else "Cannot divide by zero"
    }

    return result
```

Amazon Q Tip 1/3: Start typing to get suggesti

Event JSON

```
{
    "num1": 10,
    "num2": 5
}
```

```
PROBLEMS  OUTPUT  CODE REFERENCE LOG  TERMINAL
Status: Succeeded
Test Event Name: test-event

Response:
{
  "addition": 15,
  "subtraction": 5,
  "multiplication": 50,
  "division": 2
}

The area below shows the last 4 KB of the execution log.
```

String operations:-

lambda_function.py

```python
def lambda_handler(event, context):
    text = event["text"]

    result = {
        "uppercase": text.upper(),
        "lowercase": text.lower(),
        "reverse": text[::-1],
        "character_count": len(text),
        "word_count": len(text.split())
    }

    return result
```

Event JSON

```
{
  "text": "Hello AWS Lambda"
}
```

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

```
Status: Succeeded
Test Event Name: test-event

Response:
{
  "uppercase": "HELLO AWS LAMBDA",
  "lowercase": "hello aws lambda",
  "reverse": "adbmaL SWA olleH",
  "character_count": 16,
  "word_count": 3
}

The area below shows the last 4 KB of the execution log.
```

CloudWatch Logs:-

lambda_function.py

```python
def lambda_handler(event, context):
    print("Received event:", event)

    text = event["text"]
    print("Processing text:", text)

    result = {
        "uppercase": text.upper(),
        "lowercase": text.lower(),
        "reverse": text[::-1],
        "character_count": len(text),
        "word_count": len(text.split())
    }

    print("Final output:", result)
    return result
```

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

```
Status: Succeeded
Test Event Name: test-event

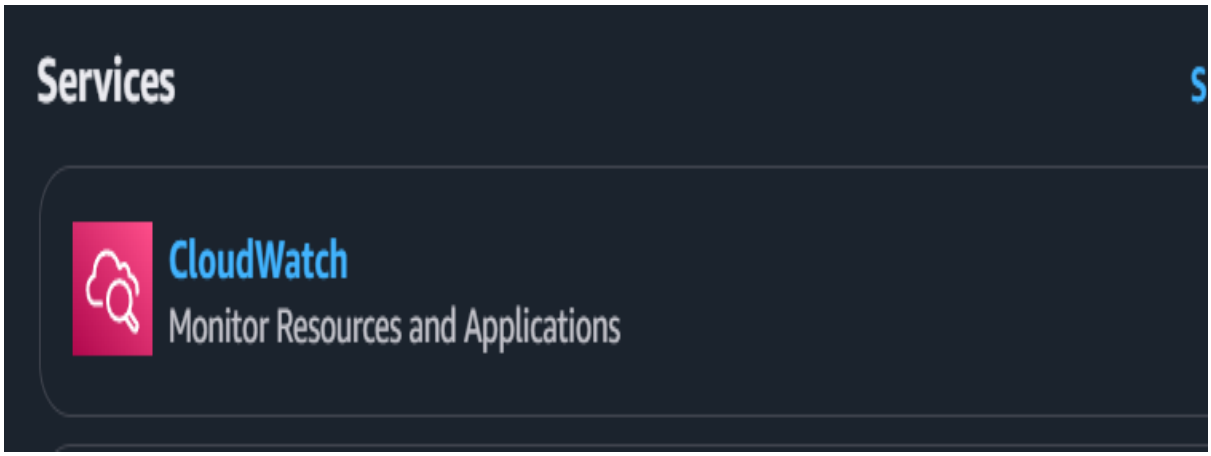
Response:
{
  "uppercase": "HELLO AWS LAMBDA",
  "lowercase": "hello aws lambda",
  "reverse": "adbmaL SWA olleH",
  "character_count": 16,
  "word_count": 3
}
```

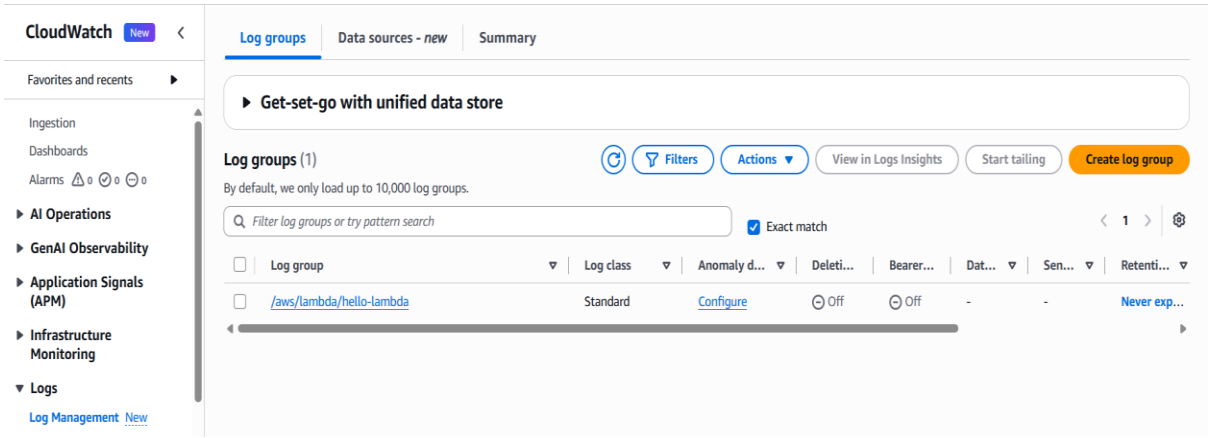
```
The area below shows the last 4 KB of the execution log.

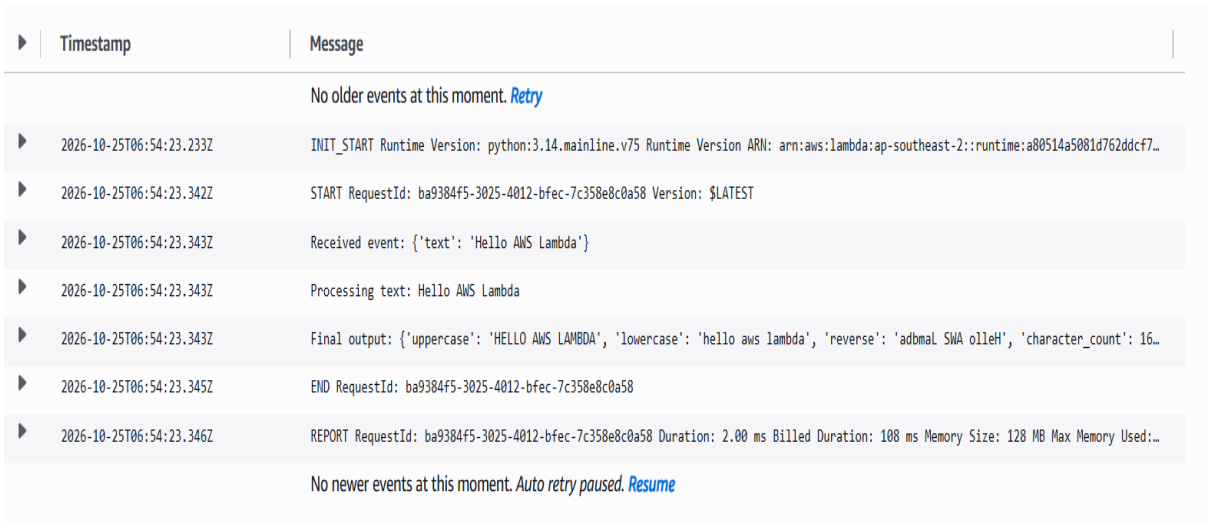
Function Logs:
START RequestId: ba9384f5-3025-4012-bfec-7c358e8c0a58 Version: $LATEST
Received event: {'text': 'Hello AWS Lambda'}
Processing text: Hello AWS Lambda
Final output: {'uppercase': 'HELLO AWS LAMBDA', 'lowercase': 'hello aws lambda', 'reverse': 'adbmaL SWA olleH', 'cha
END RequestId: ba9384f5-3025-4012-bfec-7c358e8c0a58
REPORT RequestId: ba9384f5-3025-4012-bfec-7c358e8c0a58  Duration: 2.00 ms   Billed Duration: 108 ms Memory Size: 128
Duration: 105.76 ms

Request ID: ba9384f5-3025-4012-bfec-7c358e8c0a58
```

- Cloudwatch logs

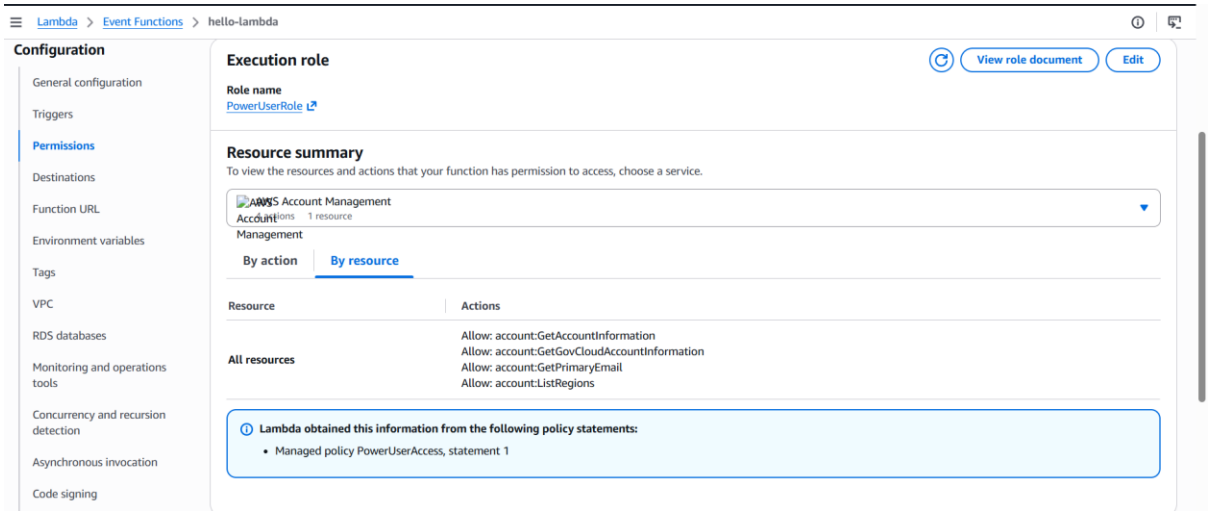




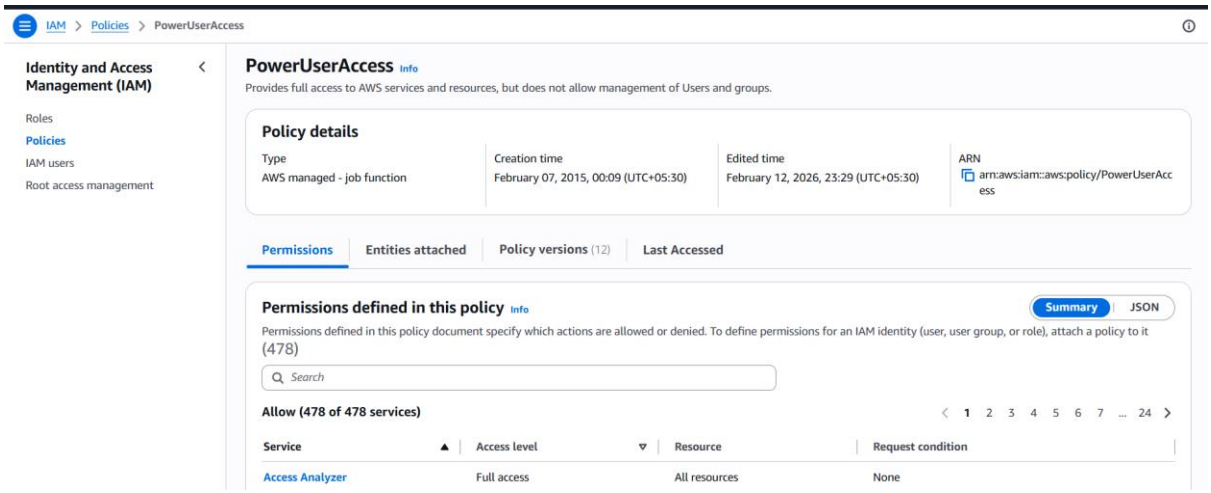


IAM Execution Role:-

- Lambda execution role from the configuration tab



- Attached policies are



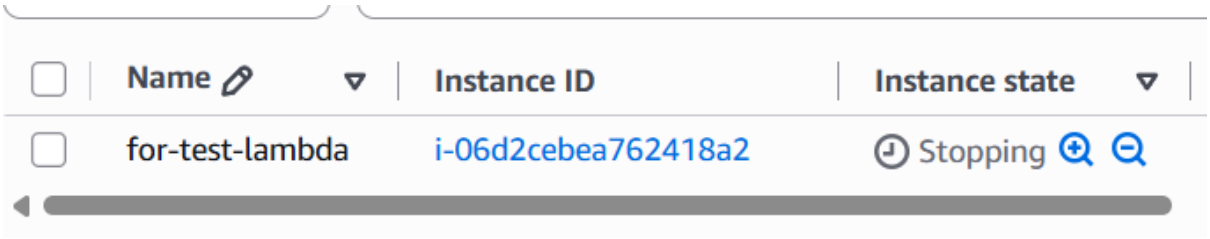
Access Analyzer	Full access	All resources	None
Account	Full: List Limited: Read	All resources	None
Account access manager	Full access	All resources	None
Action Recommendations	Full access	All resources	None
Activate	Full access	All resources	None
Agent Registry	Full access	All resources	None
AI Operations	Full access	All resources	None
Alexa for Business	Full access	All resources	None
AMP	Full access	All resources	None
Amplify	Full access	All resources	None
Amplify Admin	Full access	All resources	None
Amplify UI Builder	Full access	All resources	None
Apache Kafka APIs for MSK	Full access	All resources	None
API Gateway	Full access	All resources	None
API Gateway V2	Full access	All resources	None

App Mesh	Full access	All resources	None
App Mesh Preview	Full access	All resources	None
App Runner	Full access	All resources	None
App Studio	Full access	All resources	None
App2Container	Full access	All resources	None

- Execution role: PowerUserRole
- Attached policy shown: PowerUserAccess
- Policy type: AWS-managed policy.
- Purpose: Gives the Lambda function permissions to access AWS services.
- Access level: Broad access to AWS services and resources.
- Limitation: It does not allow management of IAM users and groups.
- Security: In production, follow the principle of least privilege by granting only the permissions required.

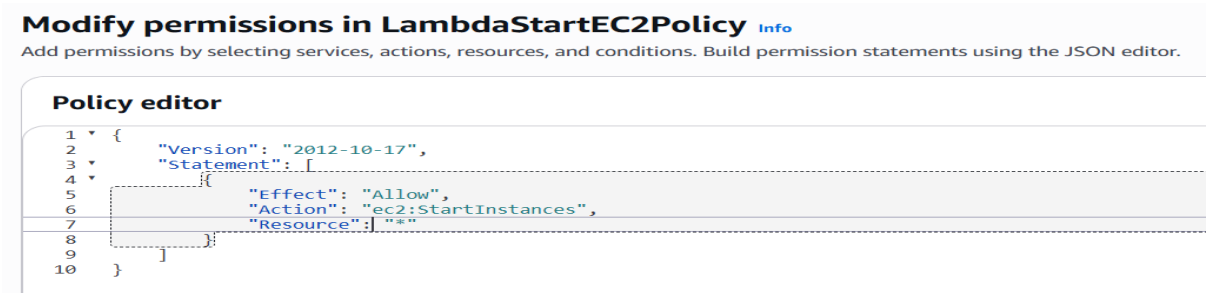
Start an EC2 Instance:-

- For this first want to launch a EC2 instance and stop it.



- Give lambda permission to start EC2

- Open Lambda → hello-lambda.
- Select Configuration → Permissions.
- Under Execution role, click `PowerUserRole`.
- In IAM, open the role's Permissions tab.
- Click Add permissions → Create inline policy.
- Select the JSON editor and paste:



- policy name LambdaStartEC2Policy.

- Write the Lambda Python code

lambda_function.py

```python

ec2 = boto3.client("ec2")

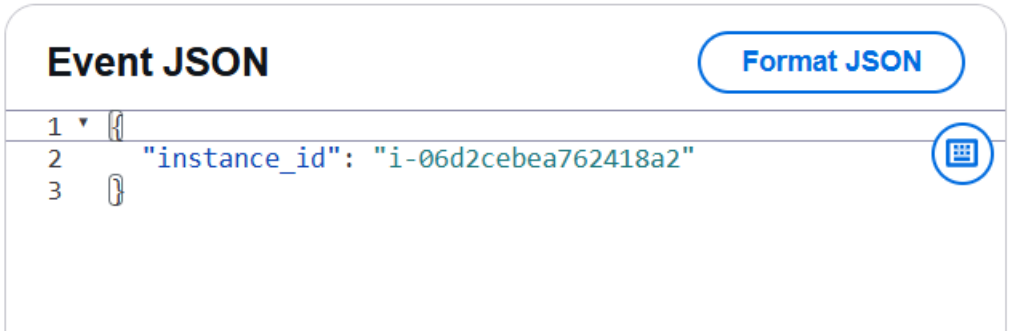
def lambda_handler(event, context):
    instance_id = event["instance_id"]

    response = ec2.start_instances(
        InstanceIds=[instance_id]
    )

    return {
        "message": "EC2 instance start requested successfully",
        "instance_id": instance_id,
        "response": response["StartingInstances"]
    }

```

- Create the test event JSON



- Result

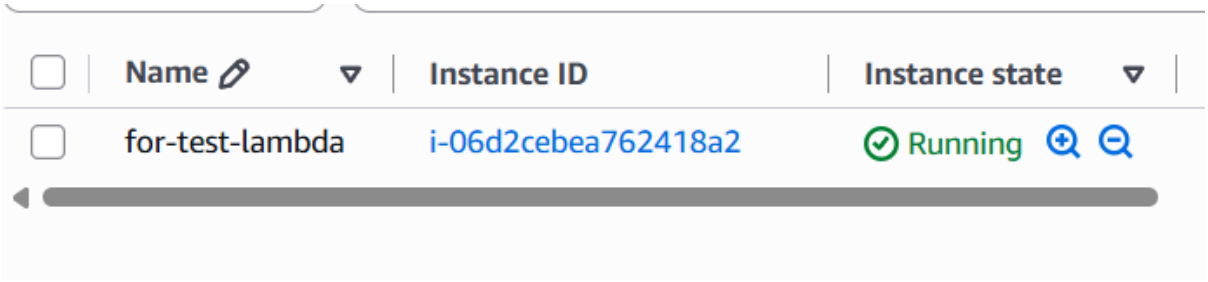
PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

```
Status: Succeeded
Test Event Name: test-event

Response:
{
  "message": "EC2 instance start requested successfully",
  "instance_id": "i-06d2cebea762418a2",
  "response": [
    {
      "InstanceId": "i-06d2cebea762418a2",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}

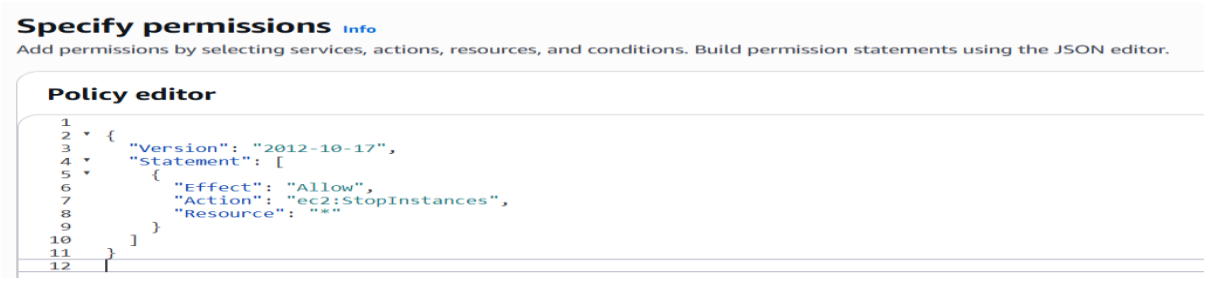
The area below shows the last 4 KB of the execution log.
```

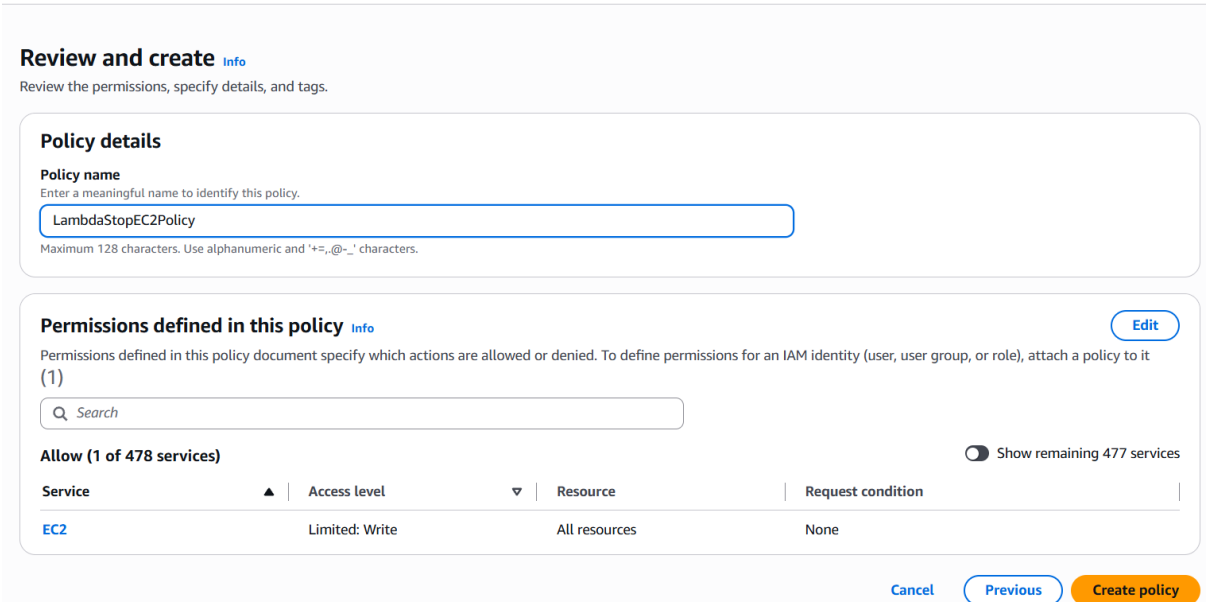
- After that EC2-instance state become running



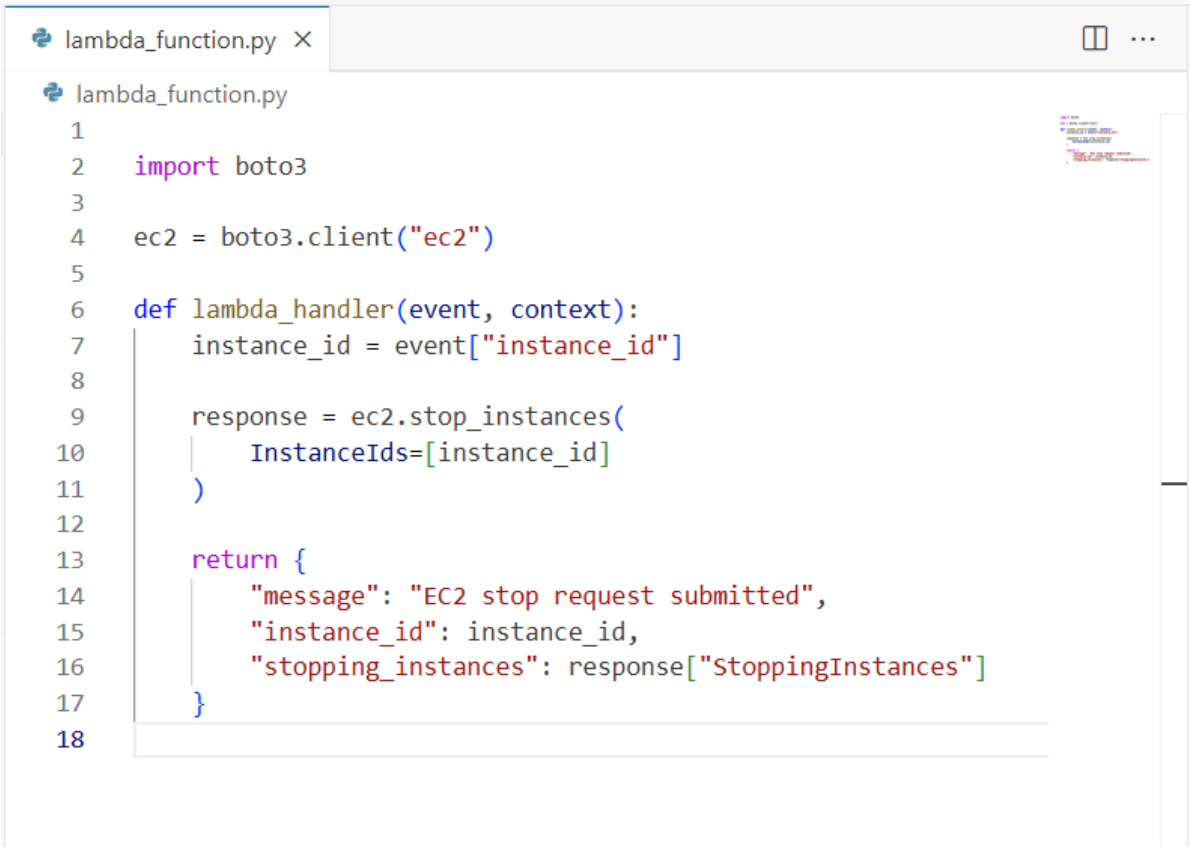
- **Stop an EC2 Instance Using AWS Lambda:-**

- Add lambda permission to stop EC2

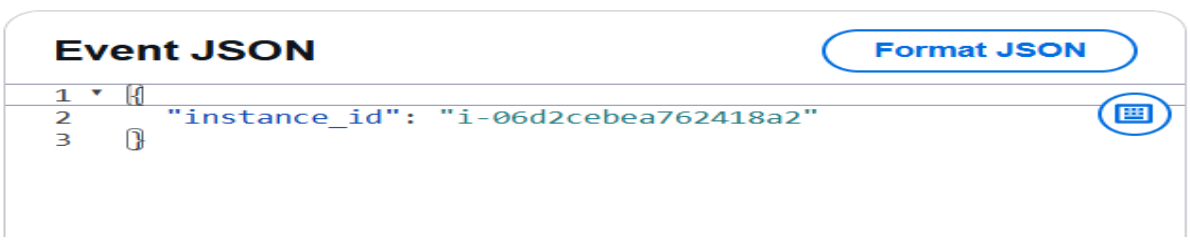




- Update python for stopping



- json



- result:

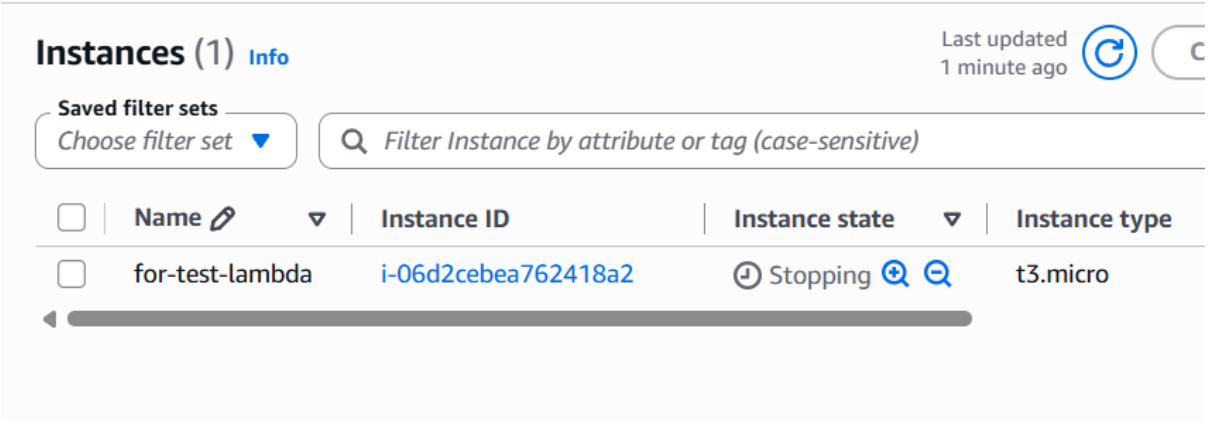
PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

```
Status: Succeeded
Test Event Name: test-event

Response:
{
  "message": "EC2 stop request submitted",
  "instance_id": "i-06d2cebea762418a2",
  "stopping_instances": [
    {
      "InstanceId": "i-06d2cebea762418a2",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}

The area below shows the last 4 KB of the execution log.
```

- instance is stopped



- **Mini Project — Start and Stop EC2 Using AWS Lambda**
- To create a single AWS Lambda function that can start or stop an EC2 instance based on the action provided in the test event.

lambda_function.py

```python
import boto3

ec2 = boto3.client("ec2")

def lambda_handler(event, context):
    action = event.get("action")
    instance_id = event.get("instance_id")

    if not instance_id:
        return {"error": "instance_id is required"}

    print("Received action:", action)
    print("Instance ID:", instance_id)

    if action == "start":
        ec2.start_instances(InstanceIds=[instance_id])
        message = "EC2 start request submitted"

    elif action == "stop":
        ec2.stop_instances(InstanceIds=[instance_id])
        message = "EC2 stop request submitted"

    else:
        return {"error": "Invalid action. Use start or stop."}

    print(message)
```

Amazon Q

- Test 1: Start EC2 Instance



```
Response:
{
  "action": "start",
  "instance_id": "i-06d2cebea762418a2",
  "message": "EC2 start request submitted"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043 Version: $LATEST
Received action: start
Instance ID: i-06d2cebea762418a2
EC2 start request submitted
END RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043
REPORT RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043  Duration: 1116.48 ms
Duration: 492.37 ms
```

Name	Instance ID	Instance state	Instance type
for-test-lambda	i-06d2cebea762418a2	Running	t3.micro

- Test 2: Stop EC2 Instance

Event JSON

```
{
  "action": "stop",
  "instance_id": "i-06d2cebea762418a2"
}
```

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

```
Status: Succeeded
Test Event Name: test-event

Response:
{
  "action": "stop",
  "instance_id": "i-06d2cebea762418a2",
  "message": "EC2 stop request submitted"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 62551578-89f4-4762-a140-589bcbfe08ee Version: $LATEST
Received action: stop
Instance ID: i-06d2cebea762418a2
EC2 stop request submitted
END RequestId: 62551578-89f4-4762-a140-589bcbfe08ee
REPORT RequestId: 62551578-89f4-4762-a140-589bcbfe08ee  Duration: 356.94 m

Request ID: 62551578-89f4-4762-a140-589bcbfe08ee
```

Instance summary for i-06d2cebea762418a2 (for-test-lambda) Info

Updated less than a minute ago

Instance ID
i-06d2cebea762418a2

IPv6 address
–

Hostname type
IP name: ip-172-31-41-1.ap-southeast-2.compute.internal

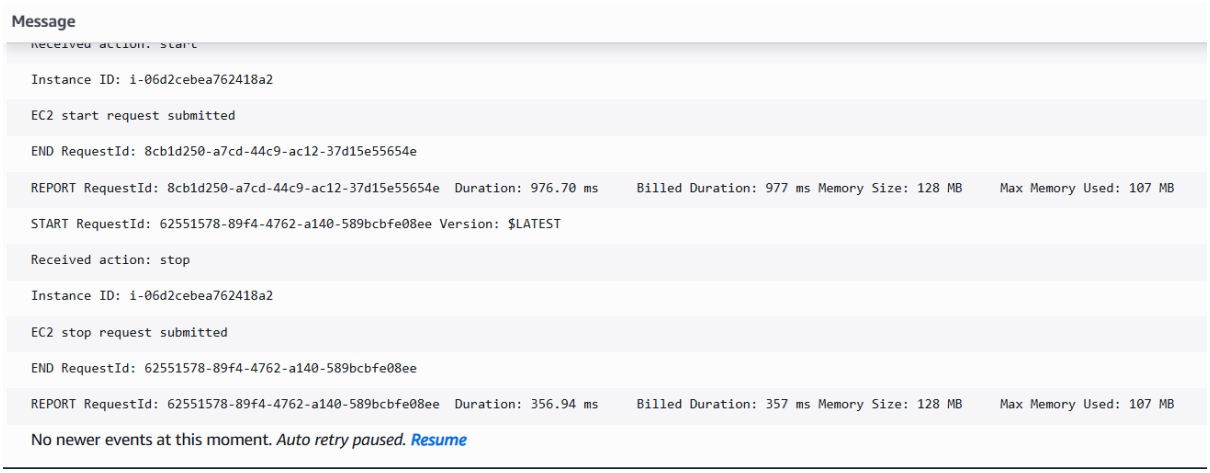
Public IPv4 address
–

Instance state
Stopped

Private IP DNS name (IPv4 only)
ip-172-31-41-1.ap-southeast-2.compute.internal

- CloudWatch Logs Verification

Message
No older events at this moment. *Retry*
INIT_START Runtime Version: python:3.14.mainline.v75 Runtime Version ARN: arn:aws:lambda:ap-southeast-2::runtime:a80514a5081d762ddcf7613643fa0106781d9c6d290a1c3536b6290ceef969a8
START RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043 Version: $LATEST
Received action: start
Instance ID: i-06d2cebea762418a2
EC2 start request submitted
END RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043
REPORT RequestId: a629a2f1-63f7-49bd-8b67-817c9686e043 Duration: 1116.48 ms Billed Duration: 1609 ms Memory Size: 128 MB Max Memory Used: 107 MB Init Duration: 492.37 ms
START RequestId: 8cb1d250-a7cd-44c9-ac12-37d15e55654e Version: $LATEST
Received action: start



CloudWatch Logs confirms that the Lambda function executed both EC2 actions.

- Start action: The logs show Received action: start and EC2 start request submitted.
- Stop action: The logs show Received action: stop and EC2 stop request submitted.
- Instance ID: The same EC2 instance was used for both operations.
- Monitoring: CloudWatch Logs helps monitor Lambda execution and troubleshoot errors.

Conclusion: The Lambda function successfully submitted both start and stop requests for the EC2 instance.